

Mục lục

1	Assignment KTLT 01	2
1.1	Đoạn code task 0	2
1.1.1	Mã code task 0	2
1.1.2	Phân tích đoạn code	3
1.2	Đoạn code cho task 1	4
1.3	Đoạn code cho task 2	4
1.3.1	Mã code task 2	4
1.3.2	Phân tích đoạn code	7
1.4	Đoạn code cho Task 3	7
1.4.1	Mã code Task 3	7
1.4.2	Phân tích đoạn code	9
1.5	Đoạn code Task 4	9
1.5.1	Mã nguồn	9
1.5.2	Phân tích đoạn code	10
1.6	Đoạn code cho Task 5	10
1.6.1	Mã nguồn của Task 5	10
1.6.2	Phân tích đoạn code	11

Chương 1

Assignment KTLT 01

1.1 Đoạn code task 0

1.1.1 Mã code task 0

```
1  #include "tay_nguyen_campaign.h"
2  const int MAX_LINES = 5;
3  const int MAX_LINE_LENGTH = 100;
4  const int NUM_LF = 17;
5  const int MAX_LF = 1000;
6  const int MAX_EXP = 600;
7  const int MAX_T = 3000;
8  const int MAX_E = 99;
9  //
10 void rangeCheck( int &x, int max){
11     if(x > max ) x = max;
12     if (x < 0 ) x = 0;
13 }
14 //
15 bool readFile(
16     const string &filename,
17     int LF1[], int LF2[],
18     int &EXP1, int &EXP2,
19     int &T1, int &T2, int &E)
20 {
21     char data[MAX_LINES][MAX_LINE_LENGTH];    //
22     int numLines = 0;
23
24     ifstream ifs(filename);                    //
25     if (!ifs.is_open())
26         //
27     {
28         return false;
29     }
30
31     while (numLines < MAX_LINES && ifs.getline(data[numLines], MAX_LINE_LENGTH))
32         //
33     {
34         numLines++;
35     }
36     ifs.close();
37
38     if (numLines < MAX_LINES)                  //
39     {
40         return false;
41     }
42
43     //
44     //
45     stringstream line1(data[0]);
46     char test;
47     if (line1.get() != '[') return false; //
48     for (int i = 0; i < NUM_LF; i++) {
49         if (!(line1 >> LF1[i])) return false;
```

```

50         if (i < NUM_LF - 1) {                                //
51             if (line1.get() != ',') return false;           //
52         }
53     }
54     if (line1.get() != ']') return false;                    //
55     if (line1 >> test) return false; //
56 //
57 stringstream line2(data[1]);
58 if (line2.get() != '[') return false;
59 for (int i = 0; i < NUM_LF; i++) {
60     if (!(line2 >> LF2[i])) return false;
61     if (i < NUM_LF - 1) {
62         if (line2.get() != ',') return false;
63     }
64 }
65 //
66 stringstream line3(data[2]);
67 if (!(line3 >> EXP1 >> EXP2) || line3 >> test) return false;
68 //
69 stringstream line4(data[3]);
70 if (!(line4 >> T1 >> T2) || line4 >> test) return false;
71 //
72 stringstream line5(data[4]);
73 if (!(line5 >> E) || line5 >> test) return false;
74 //
75 for (int i = 0; i < NUM_LF; i++) {
76     rangeCheck(LF1[i], MAX_LF);
77     rangeCheck(LF2[i], MAX_LF);
78 }
79 rangeCheck(EXP1, MAX_EXP);
80 rangeCheck(EXP2, MAX_EXP);
81 rangeCheck(T1, MAX_T);
82 rangeCheck(T2, MAX_T);
83 rangeCheck(E, MAX_E);
84 return true;
85 }
86 }

```

1.1.2 Phân tích đoạn code

1.2 Đoạn code cho task 1

```
1   int gatherForces(int LF1[], int LF2[])
2   {
3       //
4       const int weightForces[] = {1,2,3,4,5,7,8,9,10,12,15,18,20,30,40,50,70};
5       int totalForces_1 = 0, totalForces_2 = 0;
6
7       for (int i = 0; i < 17; i++) {
8           totalForces_1 += LF1[i] * weightForces[i];
9           totalForces_2 += LF2[i] * weightForces[i];
10      }
11
12      return (totalForces_1 + totalForces_2);
13  }
```

1.3 Đoạn code cho task 2

1.3.1 Mã code task 2

```
1  bool isdigit(char ch) {          //
2      return ch >= '0' && ch <= '9';
3  }
4  bool isalpha(char c) {          //
5      return (c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z');
6  }
7  bool islower(char c) {          //
8      return (c >= 'a' && c <= 'z');
9  }
10 // Task 2
11 string determineRightTarget(const string &target)
12 {
13
14     int numbers[3] = {0};
15     int m = 0;
16     string numberID = "";
17     //
18     //
19     for (char ch : target) {      //
20         if (isdigit(ch)) {
21             numberID += ch;        //
22         } else {
23             if (!numberID.empty()) { //
24                 int num = stoi(numberID);
25                 if (num > 100) return "INVALID";
26                 if (m >= 3) return "INVALID";
27                 numbers[m++] = num;
28                 numberID = "";
29             }
30         }
31     }
32
33     //
34     if (!numberID.empty()) {
35         int num = stoi(numberID); //
36         //
37         if (num > 100) return "INVALID";
38         if (m >= 3) return "INVALID";
39         numbers[m++] = num;
40     }
41
42     if (m == 0) return "INVALID";
43
44     int ID = -1;
45     if (m == 1) {
46         int num = numbers[0];
47         if (num >= 3 && num <= 7) {
```

```

48         ID = num;
49     } else if (num >= 0 && num <= 2) {
50         return "DECOY";
51     } else {
52         return "INVALID";
53     }
54 } else if (m == 2) {
55     ID = ((numbers[0] + numbers[1]) % 5) + 3;
56 } else { // m == 3
57     int maxNum = (numbers[0] > numbers[1]) ? ((numbers[0] > numbers[2]) ? numbers[0] :
58         numbers[2]) : ((numbers[1] > numbers[2]) ? numbers[1] : numbers[2]);
59     ID = (maxNum % 5) + 3;
60     if (ID == 3) return "Buon_Ma_Thuot";
61     if (ID == 4) return "Duc_Lap";
62     if (ID == 5) return "Dak_Lak";
63     if (ID == 6) return "National_Route_21";
64     if (ID == 7) return "National_Route_14";
65     return "INVALID";
66 }
67 const string TARGETS[] = {"Buon_Ma_Thuot", "Duc_Lap", "Dak_Lak", "National_Route_21", "
68     National_Route_14"};
69 const int TARGETS_SIZE = 5;
70 //
71 string reverseString(const string &message) {
72     string result = message;
73     int len = message.size();
74     \
75     for (int i = 0; i < len / 2; i++) {
76         char tempo = result[i]; \
77         \
78         result[i] = result[len - 1 - i]; \
79         \
80         \
81         result[len - 1 - i] = tempo;
82         \
83         \
84     }
85
86     bool newWord = true;
87     for (int i = 0; i < len; i++) {
88         if (result[i] == '_') {
89             newWord = true;
90         } else {
91             if (newWord) {
92                 if (islower(result[i])) result[i] -= ('a' - 'A');
93                 newWord = false;
94             } else {
95                 if (isalpha(result[i]) && !islower(result[i])) result[i] += ('a' - 'A');
96             }
97         }
98     }
99
100     return result;
101 }
102
103 // Kim tra xem th ng đ i p c n m trong danh s ch m c ti u kh ng
104 bool checkTarget(const string &message) {
105     for (int i = 0; i < TARGETS_SIZE; i++) {
106         if (message == TARGETS[i]) {
107             return true;
108         }
109     }
110     return false;
111 }
112
113 //
114 bool ValidMessageCheck(const string &message) {
115     if (message.empty()) return false; //

```

```

116 //
117 for (char c : message) {
118     if (!(isalpha(c) || isdigit(c) || (c=='_')) ) {
119         return false;
120     }
121 }
122 return true;
123 }
124
125 //
126 string caesarCipherSolution(const string &message, int shift) {
127     string result = message;
128     int len = message.size();
129     //
130     for (size_t i = 0; i < message.size(); i++) {
131         //
132         //
133         char c = message[i];
134         if (isdigit(c) || c == '_') {
135             continue;
136         }
137         if (isalpha(c)) { //
138             char tempo = islower(c) ? 'a' : 'A'; //
139             result[i] = (c - tempo + shift) % 26 + tempo;
140         }
141     }
142     \\
143     \\
144     \\
145     bool newWord = true;
146     for (int i = 0; i < len; i++) { //
147         if (result[i] == '_') {
148             newWord = true;
149         } else {
150             if (newWord) {
151                 if (islower(result[i])) result[i] -= ('a' - 'A'); \\
152                 //
153                 newWord = false;
154             } else {
155                 if (isalpha(result[i]) && !islower(result[i])) result[i] += ('a' - 'A');
156                 \\
157                 \\
158             }
159         }
160     }
161     return result;
162 }
163 //
164 string decodeTarget(const string &message, int EXP1, int EXP2) {
165     if (!ValidMessageCheck(message)) return "INVALID";
166     string messageTarget;
167     if (EXP1 >= 300 && EXP2 >= 300) {
168         int shift = (EXP1 + EXP2) % 26;
169         messageTarget = caesarCipherSolution(message, shift);
170     } else {
171         messageTarget = reverseString(message);
172     }
173
174     if (checkTarget(messageTarget)) {
175         return messageTarget;
176     }
177     return "INVALID";
178 }

```

1.3.2 Phân tích đoạn code

```

1 // Task 3
2 int roundUpNumber(double x){
3     int number = static_cast<int>(x); //
4     //
5     double floatingNumber = x - number;
6     //
7     if (abs(floatingNumber) < 1e-7){ //
8         return number;
9     }
10    return static_cast<int>(ceil(x)); // N u   kh   ng ,   l   m   tr   n   l   n
11    //
12    //
13 }
14 void manageLogistics(int LF1, int LF2, int EXP1, int EXP2, int &T1, int &T2,int E) {
15     if(E >= 60 && E <= 99) return;
16     if(E == 0) {
17         if(LF1<0) LF1 = 0;
18         if(LF2 < 0) LF2 = 0;
19         if ((LF1 + LF2)==0) return;
20         double total = T1 + T2; //
21         double rate_1 = static_cast<double> (LF1) /(LF1+LF2); //
22         //
23         //
24         double rate_2 = static_cast<double> (LF2) / (LF1 + LF2);
25         double deltaT1 = rate_1 * (T1 + T2) * (1 + (EXP1 -EXP2) / 100.0 ) ;
26         //
27         //
28         //
29         int FindT1 = roundUpNumber(deltaT1);
30         double deltaT2 = total - deltaT1; //
31         //
32         int FindT2 = roundUpNumber(deltaT2);
33         T1 += FindT1;
34         T2 += FindT2;
35     } else {
36         if(E >= 1 && E <= 9) {
37             T1 = roundUpNumber(T1 - (E * 0.01 ) * T1); //
38             //
39             T2 = roundUpNumber(T2 - (E * 0.005) * T2); //
40             //
41         } else if (E >= 10 && E <= 29) {
42             T1 += E * 50; //
43             T2 += E * 50; //
44         } else if (E >= 30 && E <= 59) {
45             T1 = roundUpNumber(T1 + (E * 0.005) * T1 ); //
46             //
47             T2 = roundUpNumber(T2 + (E * 0.002 )* T2 ); //
48             //
49         }
50     }
51     rangeCheck(T1 , MAX_T);
52     rangeCheck(T2, MAX_T);
53 }

```


1.4.2 Phân tích đoạn code

```

1 // Task 4
2 int planAttack(int LF1, int LF2, int EXP1, int EXP2, int T1, int T2, int battleField
  [10][10])
3 {
4     if(LF1<0) LF1=0;
5     if(LF2<0) LF2 = 0;
6     //
7     double S = (LF1 + LF2) + (EXP1 + EXP2) * 5 +(T1 + T2) * 2;
8     \\
9     \\
10    for (int i=0;i<10;i++){
11        for (int j=0;j<10;j++){
12            double minus= (i % 2 == 0)? (battleField[i][j] * 2/3.0) : (battleField[i][j] *
              3/2.0); \\
13            \\
14            S -= minus;
15        }
16    }
17    //
18    if (abs(S - round(S)) < 1e-7) { \\
19        \\
20        return static_cast<int>(round(S));\\
21        \\
22        \\
23    }
24
25    //
26    return static_cast<int>(ceil(S));\\
27    \\
28 }

```

1.5.2 Phân tích đoạn code

1.6 Đoạn code cho Task 5

1.6.1 Mã nguồn của Task 5

```

1 // Task 5
2 int resupply(int shortfall, int supply[5][5]) {
3     // TODO
4     int arr[25];
5     int m = 0;
6     for (int i=0;i<5;i++){
7         for (int j = 0;j < 5;j++){
8             arr[m++]=supply[i][j];
9         }
10    }
11    // S p x p m n g l i c c p h n t t r o n g m n g t n h đ n l n
12    for (int i = 0; i < 24; i++) {
13        int minIndex = i; //
14        for (int j = i + 1; j < 25; j++) {
15            if (arr[j] < arr[minIndex]) { //
16                minIndex = j;
17            }
18        }
19        //
20        int temp = arr[i]; //
21        arr[i] = arr[minIndex];
22        //
23        arr[minIndex] = temp; //
24        //
25    }
26    //
27    int maxSum = arr[24] + arr[23] + arr[22] + arr[21] + arr[20];
28    if (maxSum < shortfall) return -1;
29    int minSum = INT_MAX; //
30    bool check = false;

```

```

31 //
32 for (int a = 0; a < 21; a++) {
33 //
34     for (int b = a + 1; b < 22; b++) {
35         //
36             for (int c = b + 1; c < 23; c++) {
37                 for (int d = c + 1; d < 24; d++) {
38                     for (int e = d + 1; e < 25; e++) {
39                         int sum = arr[a] + arr[b] + arr[c] + arr[d] + arr[e];
40                         if (sum >= shortfall && sum < minSum) {
41                             check = true;
42                             minSum = sum;
43                         }
44                     }
45                 }
46             }
47         }
48     }
49     if (check) return minSum;
50     return -1;
51 }

```

1.6.2 Phân tích đoạn code